

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación y Diseño Digital
Carrera de Ingeniería en Software

DOCUMENTO EVIDENCIA

Exposición con demostración práctica

“Herramientas que generan código fuente a partir de modelos UML”

Ciclo completo Model-Driven Development (MDD) aplicado al Proyecto Fin de Curso (PFC)

Herramienta seleccionada: MODELIO

Caso de aplicación:

AquaGest – Sistema para la Gestión Inteligente de una Camaronera

Asignatura:	Ingeniería de Requisitos (ISR-401)
Docente:	Ing. Gleiston Guerrero Ulloa, PhD.
Período académico:	2026–2027 PPA
Paralelo:	4to “A” Software

Integrantes:

Amagua Sacon Robyn Willian
Crespo Espinoza Kleber Obed
Rizzo Velez Edson Nagib

Repositorio GitHub de la actividad MDD (PFC-AquaGest-MDD-Equipo):

https://github.com/erizzov-boop/PFC-AquaGest-MDD-Equipo_D-

Índice

Resumen	3
1. Marco teórico	3
1.1. Model-Driven Engineering (MDE)	3
1.2. Model-Driven Architecture (MDA)	3
1.3. Model-Driven Development (MDD)	3
1.4. Transformaciones M2M y M2T	3
1.5. Metamodelo y perfiles UML	3
1.6. Forward, reverse y round-trip engineering	4
1.7. Diagramas UML admisibles como origen de la generación	4
1.8. Estado del arte	4
2. Justificación de la selección de la herramienta	4
2.1. Restricciones tecnológicas del PFC que condicionan la selección	4
2.2. Características de Modelio relevantes para AquaGest	4
2.3. Comparación con una alternativa descartada	4
2.4. Versión de la herramienta y dependencias	5
3. Descripción del subsistema del PFC modelado	5
3.1. Contexto	5
3.2. Actores involucrados en el subsistema	5
3.3. Subsistema seleccionado: Control de Estanques, Siembras y Seguimiento	5
3.3.1. Requisitos funcionales cubiertos	6
3.4. Alcance del módulo modelado en esta demostración	6
4. Modelo UML de origen	6
4.1. Origen de los datos del modelo	6
4.2. Clases del subsistema	6
4.3. Diagrama de clases de origen	7
4.3.1. Relaciones	7
4.4. Diagrama de casos de uso (complementario)	7
4.5. Diagrama de estados (complementario)	8
4.6. Estereotipos y perfiles a aplicar	8
4.7. Validación sintáctica y semántica del modelo	9
4.8. Archivo nativo del modelo	9
5. Configuración del mecanismo de generación	9
5.1. Generador seleccionado	9
5.2. Decisiones de mapeo modelo-código previstas	10
5.3. Archivos de configuración o plantilla	10
6. Proceso de generación	10

6.1. Invocación del generador	11
6.2. Árbol del proyecto generado	11
6.3. Tiempos de generación	12
7. Verificación del código generado	12
7.1. Evidencia de compilación y ejecución	12
7.2. Correspondencia modelo → código	12
7.3. Limitaciones observadas del generador	14
8. Cierre del ciclo (<i>roundtrip</i> controlado)	14
8.1. Cambio propuesto en el modelo	14
8.2. Evidencia antes/después	15
8.3. Manejo de bloques protegidos	15
8.4. Trazabilidad modelo → código tras el cambio	16
9. Reparto del trabajo por integrante	17
9.1. Commits diferenciados por integrante	17
10. Conclusiones	19
10.1. Valor del ciclo MDD para el PFC AquaGest	19
10.2. Aprendizajes obtenidos	19
10.3. Limitaciones	19
Referencias	20

Resumen

Este documento evidencia la exposición-demostración “Herramientas que generan código fuente a partir de modelos UML” de Ingeniería de Requisitos (ISR-401), UTEQ. El problema abordado es la ausencia, en el proceso actual del equipo, de un mecanismo automatizado que derive código Java desde los modelos UML del Proyecto Fin de Curso (PFC) **AquaGest**, sistema de gestión inteligente para una camaronera. La herramienta seleccionada es **Modelio** (código abierto), con su módulo Java Designer.

El caso de aplicación es el subsistema **Control de Estanques, Siembras y Seguimiento**, modelado con seis clases (**Estanque**, **Siembra**, **Empleado**, **SobrecargaSiembra**, **Grameo**, **AlertaCrecimiento**) sobre RF-01 a RF-08 y RF-17, complementado con diagramas de casos de uso y de estados. El equipo construyó y validó el modelo (0 errores, 0 advertencias), generó código Java para las seis clases, verificó directamente el contenido de dos de ellas – confirmando que el generador respeta la navegabilidad y multiplicidad de las asociaciones del modelo –, y ejecutó un *roundtrip* controlado que confirmó que el generador sobrescribe el código sin preservar cambios manuales. Quedan pendientes: inspeccionar las cuatro clases restantes, el registro de commits por integrante y la exposición en vivo.

1 Marco teórico

1.1 Model-Driven Engineering (MDE)

El paradigma de Ingeniería Dirigida por Modelos (*Model-Driven Engineering*, MDE) plantea que los modelos son artefactos de primera clase del desarrollo: el modelo es la fuente autoritativa del sistema, y las implementaciones se derivan de él mediante transformaciones automatizadas en lugar de escribirse manualmente [1, 2].

1.2 Model-Driven Architecture (MDA)

La Model-Driven Architecture (MDA), iniciativa del *Object Management Group* (OMG), estructura el desarrollo en tres niveles: **CIM** (*Computation Independent Model*, dominio y negocio), **PIM** (*Platform Independent Model*, lógica independiente de plataforma) y **PSM** (*Platform Specific Model*, ligado a una plataforma concreta), vinculados por transformaciones formales [3].

1.3 Model-Driven Development (MDD)

El MDD es la aplicación práctica de MDE al ciclo de vida del software: los modelos guían la construcción del sistema y el código se obtiene, total o parcialmente, mediante generadores automáticos [4, 5]. A diferencia de MDA, MDD no exige necesariamente la pila completa de especificaciones OMG.

1.4 Transformaciones M2M y M2T

- **Modelo-a-modelo (M2M)**: produce un modelo destino a partir de uno de origen (p. ej. PIM → PSM). Estándar OMG: QVT (*Query/View/Transformation*) v1.3 [6].
- **Modelo-a-texto (M2T)**: produce artefactos textuales – código fuente, configuración, documentación – a partir de un modelo. Estándar OMG: MOFM2T v1.0 [7]. La generación de código que Modelio realiza a partir del diagrama de clases de AquaGest corresponde, conceptualmente, a esta categoría.

1.5 Metamodelo y perfiles UML

Un metamodelo define la sintaxis abstracta de un lenguaje de modelado; UML se define sobre MOF (*Meta Object Facility*), especificación vigente UML 2.5.1 [8]. Los generadores operan sobre las clases, atributos, asociaciones y estereotipos del metamodelo. Los **perfiles UML** extienden el metamodelo con estereotipos y restricciones de dominio o plataforma (p. ej. un perfil de persistencia JPA/Hibernate), que los generadores explotan para producir código específico.

1.6 Forward, reverse y round-trip engineering

Forward engineering: el código se genera a partir del modelo. *Reverse engineering*: el modelo se reconstruye a partir del código. *Round-trip engineering*: ambas direcciones se mantienen sincronizadas de forma controlada. Esta actividad cubre la generación *forward* y evidencia, de forma controlada, que un cambio en el modelo se refleja en el código tras regenerar (Sección 8).

1.7 Diagramas UML admisibles como origen de la generación

El diagrama de clases es obligatorio como base de la generación estructural. Se admiten como complemento el diagrama de casos de uso, el de estados y el de secuencia, siempre que aporten a la generación o a la comprensión del subsistema.

1.8 Estado del arte

El campo de las transformaciones de modelos cuenta con más de sesenta herramientas catalogadas [9], con retos abiertos en escalabilidad, interoperabilidad y adopción industrial de MDD [2]. La adopción de MDE en la industria es aún heterogénea [10], lo que refuerza documentar con precisión las decisiones de mapeo modelo-código de este proyecto.

2 Justificación de la selección de la herramienta

2.1 Restricciones tecnológicas del PFC que condicionan la selección

De acuerdo con la ERS de AquaGest (Secciones 2.5, 2.6 y 3.4), el PFC establece las siguientes restricciones relevantes para elegir una herramienta MDD [11]:

- Arquitectura cliente-servidor con API REST (RST-05).
- Base de datos relacional, PostgreSQL o MySQL, sin motores NoSQL como almacenamiento principal (RST-01).
- Priorización de **software libre y de código abierto** por restricción presupuestaria (RST-04, Sección 2.6).
- Backend orientado a un lenguaje de propósito general con soporte maduro de frameworks web y ORM (la Sección 2.5 de la ERS no fija un lenguaje único; el equipo ratificó **Java** como lenguaje objetivo definitivo para esta actividad, ver más adelante en esta misma sección, “Versión de la herramienta y dependencias”).

2.2 Características de Modelio relevantes para AquaGest

Modelio es un entorno de modelado desarrollado por Modeliosoft (París); su núcleo es de código abierto (GPLv3 desde 2011), y sus módulos de extensión se distribuyen bajo Apache License 2.0 [12]. Esta condición de herramienta libre satisface la restricción presupuestaria RST-04 del PFC, a diferencia de alternativas comerciales como Enterprise Architect o Visual Paradigm.

Modelio soporta de forma nativa UML2, BPMN2 y ArchiMate, y es extensible mediante módulos del *Modelio Store* [13]. El módulo oficial **Java Designer / SD Java** genera código Java a partir de diagramas de clases UML, con sincronización modelo-código en modo dirigido por modelo o *round-trip*, integrándose con IDEs como Eclipse, NetBeans o IntelliJ [14]. Modelio soporta además generación y reingeniería para **C++**, **C#** y **SQL** [15], lo que permitiría en una futura extensión generar el esquema relacional (SQL) desde el mismo diagrama de clases, coherente con RST-01 (base de datos relacional). Modelio es además multiplataforma (Linux, Windows, macOS) [12], sin costos de licenciamiento adicionales para el equipo.

2.3 Comparación con una alternativa descartada

Como contraste, se evaluó también **Papyrus + Acceleo** (ecosistema Eclipse) como alternativa. Papyrus/Acceleo tiene la ventaja de implementar directamente el estándar OMG MOFM2T mediante plantillas **.mtl** explícitas, lo que ofrece mayor control fino sobre el mapeo modelo-código. Sin

embargo, se descartó para esta demostración porque exige una curva de aprendizaje mayor (instalación de un stack Eclipse completo, aprendizaje del lenguaje Acceleo) en comparación con los generadores ya empaquetados de Modelio, y porque el equipo no cuenta con experiencia previa en Acceleo, lo que habría comprometido el tiempo disponible para completar el ciclo MDD dentro del cronograma de la actividad.

2.4 Versión de la herramienta y dependencias

- **Versión de Modelio utilizada:** 5.4 (Modelio Open Source), confirmada en la barra de título de la herramienta (Figuras 4, 7).
- **Módulo de generación instalado:** Modelio Java Designer (SD Java), instalado desde el Modelio Store – versión: 5.4.
- **Lenguaje objetivo definitivo:** Java estándar (POJOs). Esta actividad se limita a la generación de clases Java a partir del modelo UML, sin persistencia (no se usa JPA/Hibernate ni ningún ORM); la persistencia hacia la base de datos relacional del PFC (RST-01) se resolverá en una fase de diseño detallado posterior, fuera del alcance de esta demostración MDD.
- **SDK/compilador requerido en el entorno del generado:** JDK 17 (LTS).

3 Descripción del subsistema del PFC modelado

3.1 Contexto

AquaGest es el sistema de información del Proyecto Fin de Curso (PFC) desarrollado para la Compañía Biofina (Camaronera El Guasmo), destinado a reemplazar el registro manual en papel de las actividades productivas por una plataforma web con captura de datos vía sensores IoT y un módulo de inteligencia artificial para análisis predictivo [11]. El sistema completo abarca diez módulos funcionales (Gestión de Estanques, Control de Siembras, Monitoreo del Agua, Alimentación, Historial Sanitario, Gestión de Empleados, Inventario, Mantenimiento, Gestión de Cosechas, Reportes e Inteligencia Artificial), con 25 requisitos funcionales formalizados (RF-01 a RF-25).

3.2 Actores involucrados en el subsistema

- **Administrador / Coordinador:** registra estanques, da de alta al personal y valida la capacidad de siembra.
- **Operario de Producción:** registra cada siembra (fecha, cantidad de larvas, proveedor, responsable) y las sobrecargas controladas cuando corresponden.
- **Técnico Acuícola:** registra los grameos semanales de seguimiento del crecimiento del camarón y da seguimiento a las alertas de crecimiento.

3.3 Subsistema seleccionado: Control de Estanques, Siembras y Seguimiento

Se selecciona como subsistema representativo **Control de Estanques, Siembras y Seguimiento**, base operativa del resto del ciclo productivo (monitoreo de agua, alimentación, sanidad y cosecha dependen de que exista un estanque con siembra activa). Cubre el ciclo completo: registro de la piscina y cálculo de capacidad, registro formal de cada siembra y su responsable, seguimiento semanal de crecimiento, con sus dos mecanismos de control – registro de sobrecargas autorizadas y generación de alertas ante crecimiento inferior al esperado.

3.3.1. Requisitos funcionales cubiertos

ID	Descripción resumida
RF-01	Registro de estanque (ID, nombre, superficie en hectáreas, estado).
RF-02	Cálculo de la capacidad máxima de siembra según la superficie del estanque.
RF-03	Registro de sobrecarga controlada de siembra con justificación.
RF-04	Actualización del estado del estanque (activo, preparación, cosecha, inactivo).
RF-05	Registro de siembra (fecha, cantidad de larvas, tipo, laboratorio, responsable).
RF-06	Registro de grameo semanal (peso promedio, fecha, observaciones).
RF-07	Historial de crecimiento del camarón por ciclo productivo.
RF-08	Alerta de crecimiento inferior al esperado.
RF-17	Registro de empleados (nombre, cédula, cargo, rol, fecha de ingreso), como responsables de las siembras del subsistema.

3.4 Alcance del módulo modelado en esta demostración

El alcance de la demostración comprende las seis clases de dominio del subsistema Control de Estanques, Siembras y Seguimiento (**Estanque**, **Siembra**, **Empleado**, **SobrecargaSiembra**, **Grameo**, **AlertaCrecimiento**), junto con sus atributos, operaciones derivadas de los RF anteriores y sus relaciones de asociación, agregación y composición (ver detalle en la Sección 4). Quedan fuera del alcance de esta demostración puntual el resto de módulos (Monitoreo del Agua, Alimentación, Historial Sanitario más allá de lo ya cubierto, Inventario, Mantenimiento, Cosecha, Reportes e Inteligencia Artificial), ya descritos en el ERS del PFC pero fuera del subsistema elegido para esta actividad.

4 Modelo UML de origen

4.1 Origen de los datos del modelo

Los atributos y operaciones de las clases presentadas a continuación se derivan directamente de los campos de *Entradas* y *Salidas* ya especificados para RF-01 a RF-08 y RF-17 en la ERS de AquaGest [11], y de las multiplicidades base ya documentadas en la Tabla 19 (“Relaciones principales y multiplicidad”) de ese mismo documento. El diagrama de clases es, por tanto, una proyección UML consistente con requisitos ya validados, elaborada como paso previo (blueprint) al modelado dentro de la herramienta Modelio.

4.2 Clases del subsistema

El subsistema Control de Estanques, Siembras y Seguimiento se modela con seis clases:

Clase	Responsabilidad dentro del subsistema
Estanque	Piscina de cultivo: identificación, superficie y estado operativo (RF-01, RF-02, RF-04).
Siembra	Ciclo de siembra iniciado en un estanque: fecha, cantidad de larvas, laboratorio de origen (RF-05).
Empleado	Responsable que ejecuta y registra una siembra (RF-17).
SobrecargaSiembra	Registro de una sobrecarga autorizada sobre una siembra, con su justificación (RF-03).
Grameo	Medición semanal de peso promedio del camarón dentro de una siembra (RF-06, RF-07).
AlertaCrecimiento	Alerta generada cuando un grameo muestra un crecimiento inferior al esperado (RF-08).

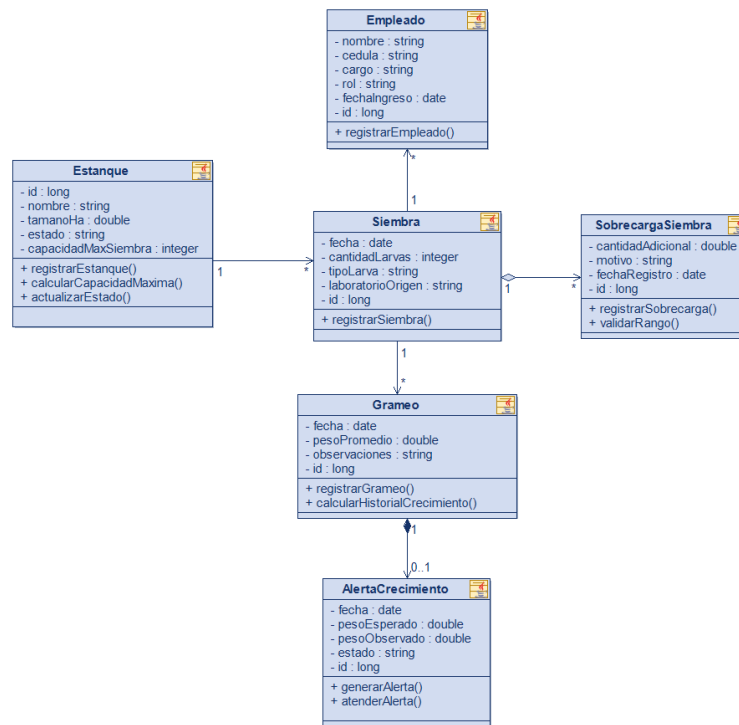


Figura 1: Diagrama de clases de origen del subsistema Control de Estanques, Siembras y Seguimiento, exportado desde Modelio.

4.3 Diagrama de clases de origen

El siguiente diagrama de clases corresponde al modelo UML desarrollado en Modelio y utilizado como base para la generación automática de código. Previamente se verificó la consistencia del modelo, incluyendo la correcta nomenclatura de todas las clases, garantizando que el código generado refleje fielmente el diseño conceptual.

4.3.1. Relaciones

Relación	Multiplicidad	Tipo
Estanque – Siembra	1 a 0..*	Asociación
Siembra – Empleado (responsables)	1 a 0..*	Asociación
Siembra – SobrecargaSiembra	1 a 0..*	Agregación
Siembra – Grameo	1 a 0..*	Asociación
Grameo – AlertaCrecimiento	1 a 0..1	Composición

Las multiplicidades de **Estanque–Siembra** y **Siembra–Grameo** provienen directamente de la Tabla 19 del ERS; el resto son una propuesta del equipo, a validar y ajustar dentro de Modelio si la herramienta o el criterio del equipo sugieren un cambio.

4.4 Diagrama de casos de uso (complementario)

El siguiente diagrama de casos de uso fue elaborado en Modelio 5.4 y representa las principales funcionalidades del subsistema, así como la interacción entre los actores y el sistema. Este diagrama complementa el modelo de clases al mostrar los procesos que cada tipo de usuario puede ejecutar, proporcionando una visión funcional del sistema antes de la generación automática de código.

Nota: Debido a que Modelio 5.4 no ofrece un elemento específico para representar el límite del sistema (Subject) en los diagramas de casos de uso, este se representó mediante un rectángulo de

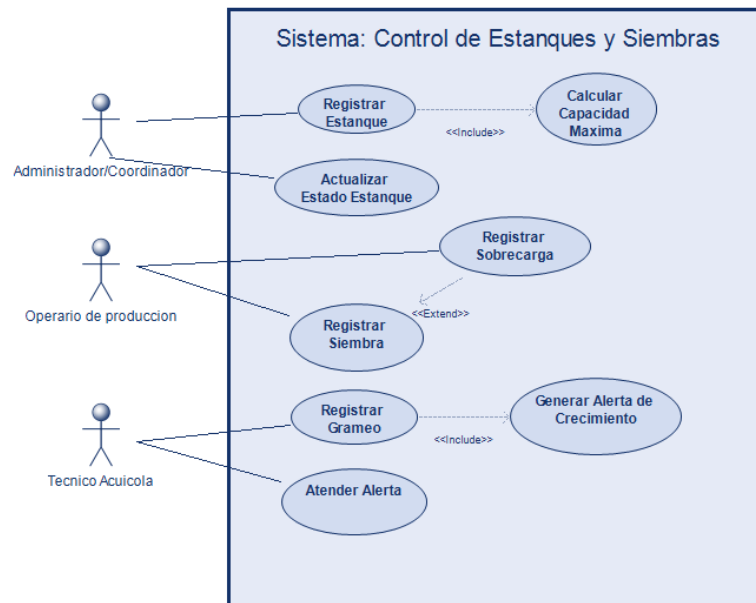


Figura 2: Diagrama de casos de uso del subsistema Control de Estanques y Siembras, elaborado en Modelio 5.4. Los tres actores corresponden a los descritos en la Sección 3.

dibujo. Esta representación es únicamente gráfica y no altera la estructura ni el significado del modelo UML.

4.5 Diagrama de estados (complementario)

El siguiente diagrama de estados fue elaborado en Modelio 5.4 y representa el ciclo de vida de la clase **Estanque**, mostrando las transiciones entre sus diferentes estados de acuerdo con los eventos definidos para el subsistema. Este diagrama complementa el modelo estructural al describir el comportamiento dinámico de la entidad durante su operación.

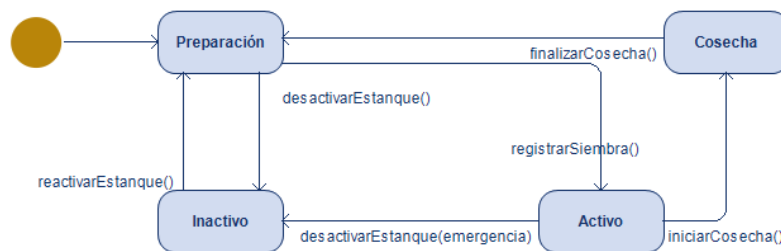


Figura 3: Diagrama de estados de la clase **Estanque**, elaborado en Modelio 5.4. Las transiciones representan el ciclo de vida definido para el subsistema Control de Estanques y Siembras, en concordancia con el requisito funcional RF-04.

4.6 Estereotipos y perfiles a aplicar

Esta actividad genera **clases Java estándar (POJOs)**, sin persistencia: no se usa JPA, Hibernate ni ningún ORM. Con ese alcance, el equipo creó dos **estereotipos locales** dentro del proyecto Modelio (no importados de un perfil externo): **<<Entity>>**, aplicado a las seis clases del subsistema para marcarlas como objetos de dominio con identidad propia, y **<<Id>>**, aplicado sobre el atributo que identifica a cada instancia dentro del modelo. Adicionalmente, se aplicó a cada clase el estereotipo **<<JavaElement>>** requerido por el módulo Java Designer para que la clase sea reconocida como candidata a generación de código Java.

4.7 Validación sintáctica y semántica del modelo

El diagrama de clases se validó dentro de Modelio mediante el panel de auditoría del modelo (*Model check*), obteniendo **0 errores y 0 advertencias** sobre las seis clases y sus relaciones (Figura 4).

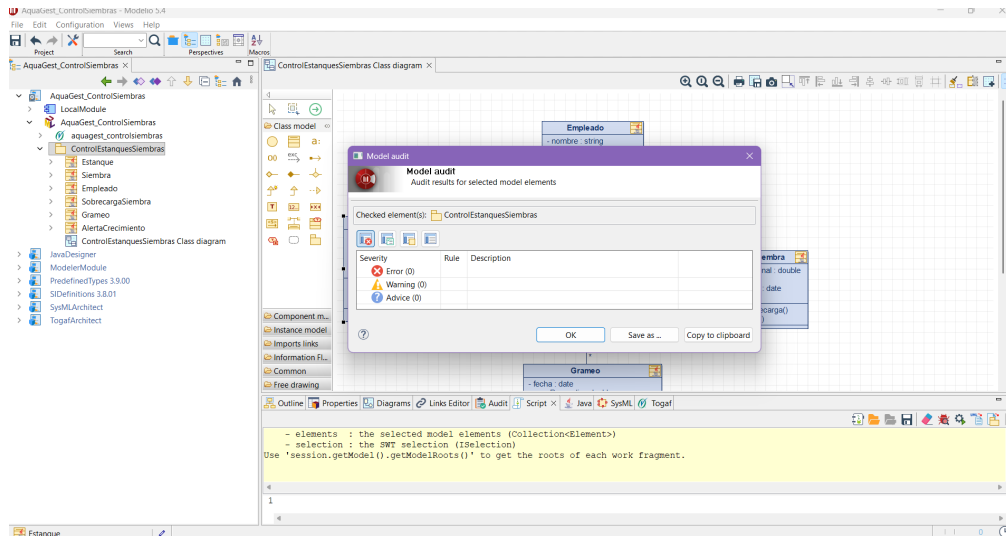


Figura 4: Panel de validación de Modelio sobre el diagrama de clases del subsistema: 0 errores y 0 advertencias.

4.8 Archivo nativo del modelo

El proyecto nativo de Modelio se exportó como archivo de proyecto (**File >Export project**) y se subió a la carpeta `modelo/` del repositorio de la actividad:

`modelo/AquaGest_ControlSiembras.zip`

Para reutilizarlo, debe importarse dentro de Modelio con **File >Import Project...** (no descomprimirse manualmente).

5 Configuración del mecanismo de generación

5.1 Generador seleccionado

Se utiliza el módulo de generación de código Java integrado en Modelio (*Modelio Java Designer / Modelio SD Java*), que produce código Java estándar directamente desde el diagrama de clases UML y mantiene la sincronización modelo-código en modo dirigido por modelo o *round-trip* [14]. El alcance es la generación de **clases Java estándar (POJOs)**, sin persistencia (no se usa JPA/Hibernate ni ningún ORM), en coherencia con la justificación de la herramienta (Sección 2).

Parámetro	Valor
Generador / módulo	Modelio Java Designer (módulo oficial) – versión: 5.4
Lenguaje objetivo	Java
Directorio de salida	MODELIOJAVA (carpeta local generada por el módulo Java Designer; ruta completa dentro del equipo de trabajo, sin persistencia adicional configurada)
Convención de nombres	<i>PascalCase</i> para clases (confirmado: <code>Estanque.java</code> , <code>SobrecargaSiembra.java</code> , etc.), <i>camelCase</i> para atributos – observado directamente en el código generado (Sección 7)
Política ante regeneración	El código se sobrescribe por completo en cada generación: Modelio Java Designer no implementa bloques de código protegido (<i>protected regions</i>) en este módulo, por lo que no preserva automáticamente la lógica de negocio agregada manualmente entre una generación y la siguiente. Confirmado al ejecutar el <i>roundtrip</i> controlado (Sección 8).

5.2 Decisiones de mapeo modelo–código previstas

Elemento del modelo	Mapeo esperado en Java
Clase Estanque, Siembra, Grameo	Clase Java pública (<code>public class</code>) con el mismo nombre.
Atributo tipado con <code>double</code> (p. ej. <code>tamanoHa: double</code>)	Campo privado Java (<code>private double tamanoHa</code>). La documentación del módulo Java Designer indica que puede generar accesores <code>get/set</code> automáticamente, aunque no se observaron en las clases inspeccionadas (Sección 7). Si en la verificación se detecta que algún cálculo requiere precisión exacta (p. ej. costos), se ajustará manualmente a <code>BigDecimal</code> tras la generación.
Asociación Estanque–Siembra (1 a 0..*)	Colección tipada en el lado “0..*” (<code>private List<Siembra>siembras</code>) y referencia simple en el lado “1”.
Operación (p. ej. <code>calcularCapacidadMaxima()</code>)	Firma de método Java generada desde la operación UML; el cuerpo del método debe completarse manualmente tras la generación, ya que la lógica de negocio no forma parte del modelo estructural.

Estas decisiones de mapeo son la interpretación esperada según el comportamiento documentado del generador de Modelio; deberán confirmarse o corregirse con la evidencia real observada en la Fase 4 del procedimiento, y las eventuales correcciones deben incorporarse a esta tabla.

5.3 Archivos de configuración o plantilla

Para esta actividad **no se editó ni personalizó** la plantilla estándar del módulo Java Designer: se utilizó el generador tal como lo distribuye Modelio, sin modificar sus reglas internas de mapeo modelo–código. Por esta razón, la carpeta `plantillas/` del repositorio no contiene archivos de plantilla personalizados. Si en una futura iteración del PFC se decide personalizar la plantilla (por ejemplo, para incorporar anotaciones JPA), esta subsección deberá actualizarse.

6 Proceso de generación

Esta sección documenta la ejecución de la generación de código y su verificación inicial, y debe completarse íntegramente con evidencia tomada durante la demostración en vivo. Se mantiene aquí la estructura prevista de los tres subapartados para que el equipo únicamente deba insertar, en su momento, las capturas y los datos reales observados.

6.1 Invocación del generador

Captura de pantalla de la invocación del generador dentro de Modelio (menú de generación de código sobre el paquete/clases del subsistema Control de Estanques y Siembras). Modelio Java Designer no despliega una consola de log: al invocar *Generate*, el proceso corre y finaliza sin mostrar ventana ni mensaje de salida; la confirmación de éxito se obtiene directamente del árbol de archivos resultante (ver subapartado siguiente).

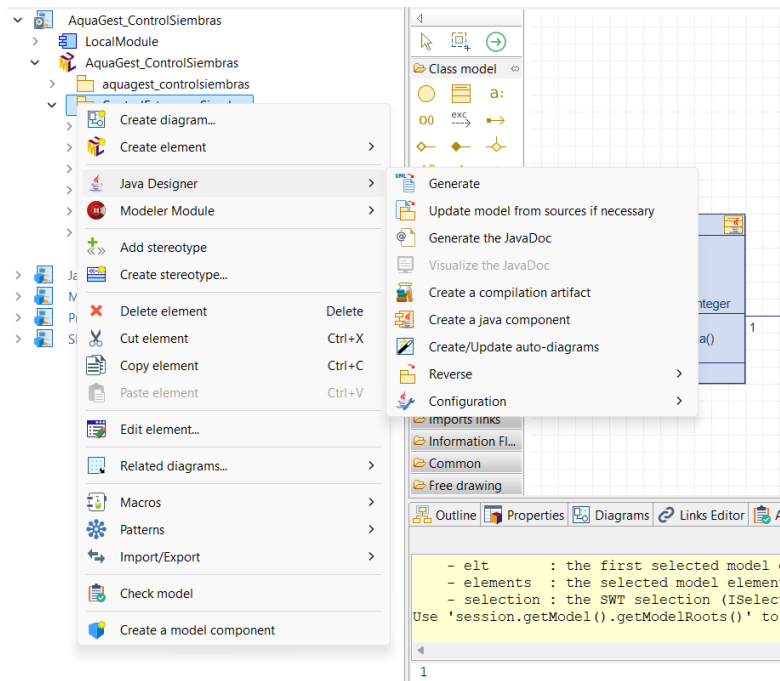


Figura 5: Invocación del generador de código en Modelio.

6.2 Árbol del proyecto generado

La generación produjo seis archivos Java, uno por cada clase del subsistema, en la carpeta de salida MODELIOJAVA (Figura 6): Estanque.java, Siembra.java, SobrecargaSiembra.java, Grameo.java, AlertaCrecimiento.java y Empleado.java.

The screenshot shows a file explorer window with the path 'Descargas > MODELIOJAVA'. It displays a table of generated Java files. The table has columns for 'Nombre' (Name), 'Fecha de modificación' (Modification Date), 'Tipo' (Type), and 'Tamaño' (Size). All files were created on 18/7/2026 at 14:39 and are 1 KB in size.

Nombre	Fecha de modificación	Tipo	Tamaño
AlertaCrecimiento.java	18/7/2026 14:39	Archivo JAVA	1 KB
Empleado.java	18/7/2026 14:39	Archivo JAVA	1 KB
Estanque.java	18/7/2026 14:39	Archivo JAVA	1 KB
Grameo.java	18/7/2026 14:39	Archivo JAVA	1 KB
Siembra.java	18/7/2026 14:39	Archivo JAVA	1 KB
SobrecargaSiembra.java	18/7/2026 14:39	Archivo JAVA	1 KB

Figura 6: Árbol de archivos .java generados por Modelio Java Designer en la carpeta MODELIOJAVA.

6.3 Tiempos de generación

Los seis archivos generados muestran la misma marca de tiempo de modificación (18/7/2026 14:39) y un tamaño uniforme de 1 KB cada uno (Figura 6), lo que evidencia que la generación del código se realizó de forma prácticamente instantánea para las seis clases del subsistema.

7 Verificación del código generado

7.1 Evidencia de compilación y ejecución

El código generado se abrió en **IntelliJ IDEA**, dentro del proyecto **MODELIOJAVA**. El inspector estático del IDE no reporta problemas sobre la clase **AlertaCrecimiento.java** (“No problems in AlertaCrecimiento.java”, Figura 7), lo que confirma que ese archivo es sintácticamente válido.

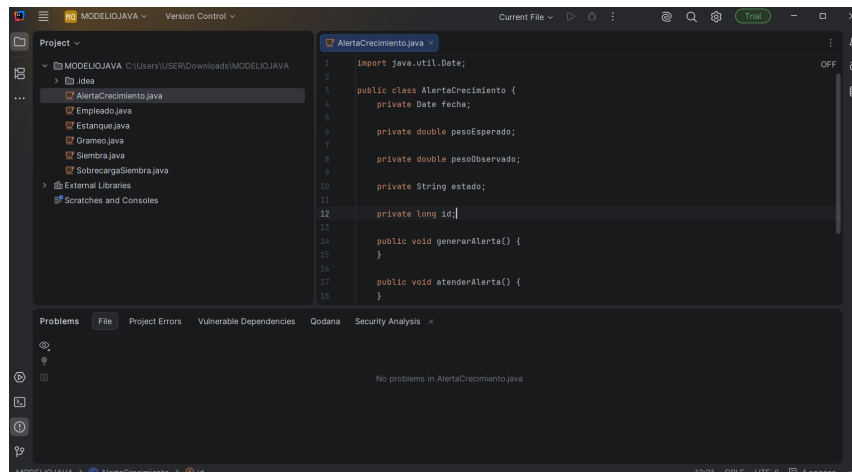


Figura 7: Clase **AlertaCrecimiento.java** generada, abierta en IntelliJ IDEA sin problemas reportados por el inspector estático.

7.2 Correspondencia modelo → código

La existencia de los seis archivos **.java** está confirmada por el árbol de archivos generado (Figura 6, Sección 6). El contenido detallado se inspeccionó directamente para dos clases: **AlertaCrecimiento.java** (Figura 7) y **Siembra.java**, esta última visible en las capturas del *roundtrip* (Figuras 9 y 10, Sección 8). Para las tres clases restantes (**Estanque**, **Empleado**, **Grameo**, **SobrecargaSiembra**) aún debe abrirse cada archivo y contrastarlo con esta tabla.

Elemento del modelo	Artefacto generado esperado	Verificado
Clase Estanque	Archivo Estanque.java	Sí (archivo existe)
Clase Siembra	Archivo Siembra.java	Sí, verificado en código: campos fecha, cantidadLarvas, tipoLarva, laboratorioOrigen, id (autogenerado).
Clase Grameo	Archivo Grameo.java	Sí (archivo existe)
Clase Empleado	Archivo Empleado.java	Sí (archivo existe)
Clase SobrecargaSiembra	Archivo SobrecargaSiembra.java	Sí (archivo existe)
Clase AlertaCrecimiento	Archivo AlertaCrecimiento.java	Sí, verificado en código: campos fecha, pesoEsperado, pesoObservado, estado, id y métodos generarAlerta(), atenderAlerta() coinciden 1:1 con el modelo.
Atributo Estanque.tamanoHa	Campo tamanoHa + accesores	Pendiente (falta abrir Estanque.java)
Asociación Estanque–Siembra	Colección List<Siembra> en Estanque.java	Pendiente – Siembra.java no contiene una referencia de vuelta a Estanque (coherente con una asociación unidireccional 1 a 0..*), pero el lado “1” (Estanque.java) no ha sido inspeccionado.
Asociación Siembra–Grameo	Colección List<Grameo> en Siembra.java	Sí, verificado en código: se generó public List<Grameo> en Siembra.java.
Asociación Siembra–Empleado (responsables)	Colección List<Empleado> en Siembra.java	Sí, verificado en código: se generó public List<Empleado> en Siembra.java, coherente con la navegabilidad Siembra → Empleado y la multiplicidad 1 a 0..* del modelo (una siembra puede tener más de un empleado responsable).
Agregación Siembra–SobrecargaSiembra	Colección List<SobrecargaSiembra> en Siembra.java	Sí, verificado en código: se generó public List<SobrecargaSiembra> en Siembra.java.
Composición Grameo–AlertaCrecimiento	Referencia AlertaCrecimiento en Grameo.java	Pendiente (falta abrir Grameo.java)
Operación registrarSiembra()	Firma de método en Siembra.java	Sí, verificado en código: existe como public void registrarSiembra() con cuerpo vacío (ver Sección 8 para la prueba de sobrescritura tras regenerar).

7.3 Limitaciones observadas del generador

A partir de la revisión directa de `AlertaCrecimiento.java` y `Siembra.java` se verificó que el generador produce correctamente la estructura básica de una clase (atributos tipados, identificador autogenerado y firmas de operación) a partir del modelo UML. Los cuerpos de las operaciones (`generarAlerta()`, `atenderAlerta()`, `registrarSiembra()`) se generaron **vacíos**: la implementación de la lógica de negocio debe desarrollarse manualmente. Este comportamiento es consistente con el funcionamiento esperado de un generador de código estructural, cuyo objetivo es producir la estructura del sistema y no la implementación de sus reglas de negocio.

Se confirmó también que la asociación Siembra–Empleado se refleja correctamente como `public List<Empleado>` en `Siembra.java`, coherente con la navegabilidad Siembra → Empleado y la multiplicidad 1 a 0..* definida en el modelo (Sección 4.3): una siembra puede tener más de un empleado responsable.

Finalmente, las validaciones propias de los requisitos funcionales, como la verificación de que la cantidad de larvas no supere la capacidad máxima de siembra establecida por el requisito funcional RF-05, no son implementadas automáticamente por Modelio y deben incorporarse manualmente durante la etapa de desarrollo.

8 Cierre del ciclo (*roundtrip* controlado)

8.1 Cambio propuesto en el modelo

Como cambio no trivial para evidenciar el cierre del ciclo MDD, se propone agregar un nuevo atributo a la clase `Siembra`: `porcentajeSupervivenciaInicial: double`, que registra el porcentaje estimado de supervivencia de las larvas al momento de la siembra (dato mencionado como relevante en el acta de entrevista con el Técnico Acuícola, Apéndice B.3 del ERS de AquaGest, aunque todavía no formalizado como RF independiente). Este cambio es representativo porque:

- No es un cambio trivial de tipo tipográfico, sino la incorporación de un dato de negocio nuevo.
- Debe reflejarse tanto en el modelo (nuevo atributo con su tipo) como en el código generado (nuevo campo).
- Permite verificar si el módulo de generación preserva el código manualmente añadido en los métodos ya completados en la Fase 5, o si lo sobrescribe.

8.2 Evidencia antes/después

- Captura del modelo **antes** del cambio (diagrama de clases con **Siembra** en su versión original).
- Captura del modelo **después** del cambio, con el nuevo atributo agregado (observaciones).

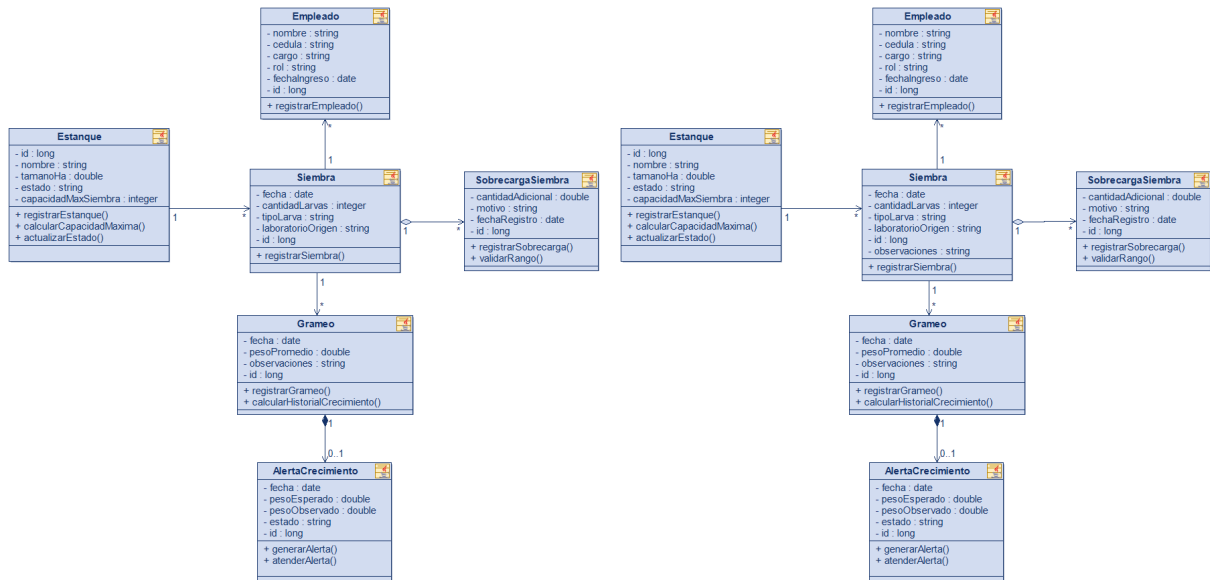


Figura 8: Modelo antes y después del cambio en **Siembra**.

- Captura del código generado **antes** de la regeneración.
- Captura del código generado **después** de la regeneración, mostrando el nuevo campo.

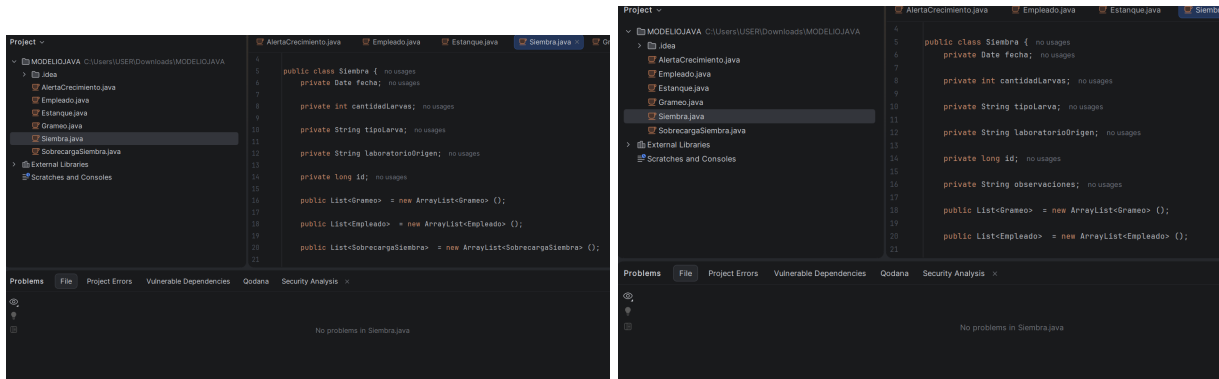


Figura 9: Código Java de **Siembra** generado por Modelio, antes y después de la regeneración con el nuevo atributo **porcentajeSupervivenciaInicial** agregado en el modelo.

8.3 Manejo de bloques protegidos

Modelio Java Designer, en el modo de generación utilizado en esta actividad, **no implementa bloques de código protegido** (*protected regions*) que preserven automáticamente la lógica de negocio agregada manualmente entre una generación y la siguiente. Como se documentó en la Sección 5 (“Política ante regeneración”), cada ejecución del generador **sobrescribe por completo** el archivo .java correspondiente con el contenido derivado del modelo.

En consecuencia, para evitar la pérdida de trabajo manual al regenerar el código, se recomienda adoptar una de las siguientes estrategias:

1. **Respaldo manual:** copiar fuera del árbol MODELIOJAVA la lógica de negocio implementada antes de regenerar el código y reincorporarla posteriormente mediante copia/pegado o utilizando un sistema de control de versiones.
2. **Separación en clases de servicio (recomendada):** trasladar la lógica de negocio a clases de servicio o utilitarias independientes, fuera del árbol de clases de dominio generado por Modelio, de modo que la regeneración de **Estanque**, **Siembra**, **Grameo**, **AlertaCrecimiento**, **SobrecargaSiembra** y **Empleado** no afecte dicha lógica. Esta estrategia resulta adecuada para el desarrollo posterior del PFC y es coherente con la arquitectura cliente-servidor por capas definida en la restricción RST-05 de la ERS.

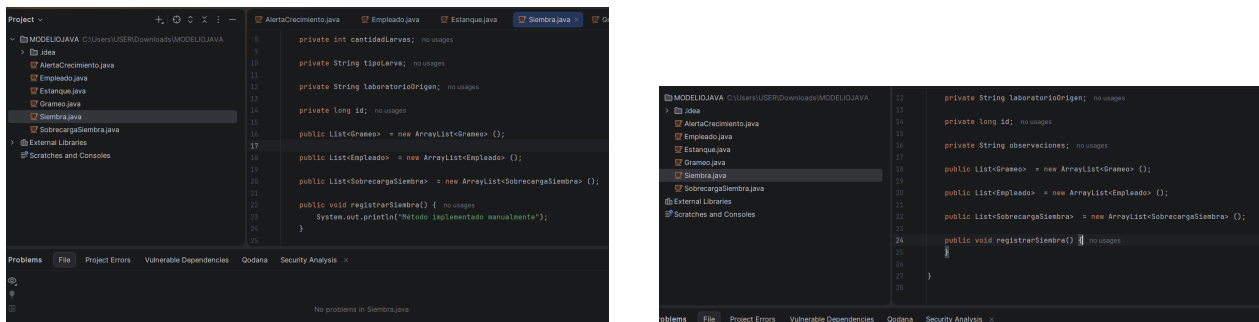


Figura 10: Código generado antes (con una modificación manual en el método `registrarSiembra()`) y después de la regeneración desde Modelio. Se observa que la modificación manual fue reemplazada por el código generado automáticamente.

La Figura 10 evidencia que, tras modificar manualmente el método `registrarSiembra()` y ejecutar nuevamente la generación de código, los cambios realizados fueron sobrescritos. Esto confirma que Modelio Java Designer, con la configuración utilizada en esta práctica, no preserva automáticamente las modificaciones manuales mediante bloques protegidos, por lo que es necesario respaldar el código o separar la lógica de negocio en clases no generadas antes de regenerar el proyecto.

8.4 Trazabilidad modelo → código tras el cambio

A pesar de que la regeneración sobrescribe las modificaciones manuales realizadas en los archivos `.java`, la trazabilidad entre el modelo UML y el código generado se mantuvo durante la práctica. Después de realizar modificaciones en el modelo de Modelio y ejecutar nuevamente la generación de código, se verificó que los cambios definidos en el modelo fueron reflejados correctamente en los archivos Java correspondientes. Asimismo, se comprobó que la correspondencia entre clases, atributos, operaciones y relaciones se conservó tras la regeneración. La Tabla presentada en la Sección 7.2 resume la verificación de dicha trazabilidad.

9 Reparto del trabajo por integrante

La distribución de tareas para esta actividad corresponde al equipo asignado a la exposición-demostración MDD, integrado por tres miembros.

Integrante	Rol en la actividad MDD	Tarea concreta asignada
Amagua Sacon Robyn Willian	Coordinación general, justificación de la herramienta	Coordina al equipo y los tiempos de la exposición, redacta la justificación de Modelio frente a las restricciones del PFC (Sección 2), y estructura/mantiene este documento evidencia en LaTeX.
Crespo Espinoza Kleber Obed	Modelado UML en Modelio y documentación LaTeX	Construye los diagramas de clases, casos de uso y estados dentro de Modelio (Sección 4), aplica estereotipos y ejecuta la validación sintáctica/semántica del modelo.
Rizzo Velez Edson Nagib	Configuración del generador, ejecución de la generación, verificación de código y cierre del ciclo	Configura los parámetros del módulo de generación (Sección 5), ejecuta el proceso de generación (Sección 6), verifica la compilación/ejecución del código generado (Sección 7) y ejecuta el <i>roundtrip</i> controlado documentado en la Sección 8.

9.1 Commits diferenciados por integrante

Se registra a continuación la lista de commits atribuibles a cada integrante.

Integrante	Contribución (mensaje del commit)	URL del commit
Amagua Sacon Robyn Willian	■ Add files via upload. Documento en LaTeX que reúne la evidencia del trabajo realizado con Modelio para generar código a partir de diagramas UML en el proyecto AquaGest. ■ Add files via upload. Archivo de bibliografía en formato BibTeX con las referencias utilizadas en el informe. ■ Código generado de la clase Siembra. ■ Código generado de la clase SobrecargaSiembra. ■ Diapositivas de Exposición de la herramienta Modelio.	■ 733bd48 ■ 6b44903 ■ 05ddb81 ■ b48b2c7 ■ 1185361
Crespo Espinoza Kleber Obed	■ Agregar secciones 3, 4 y 5 del informe. Se incorporaron las subsecciones: 3. Descripción del subsistema del PFC modelado (contexto, actores, subsistema seleccionado, requisitos funcionales cubiertos, alcance del módulo); 4. Modelo UML de origen (origen de los datos, clases, diagrama de clases con sus relaciones, diagrama de casos de uso, diagrama de estados, estereotipos y perfiles, validación sintáctica y semántica, archivo nativo); 5. Configuración del mecanismo de generación (generador seleccionado, decisiones de mapeo modelo-código, archivos de configuración o plantilla). ■ Código generado de clase Estanque. ■ Código generado de clase Grameo. ■ Agrega proyecto exportado desde Modelio. ■ Agrega conclusiones. Se añaden las subsecciones 10.1 Valor del ciclo MDD para el PFC AquaGest; 10.2 Aprendizajes obtenidos; 10.3 Limitaciones.	■ a4a0bf6 ■ fc1e8f6 ■ ade5550 ■ b6ac964 ■ 34845d3
Rizzo Vélez Edson Nagib	■ Agregar sección 6, 7, 8 y 9. Junto con las secciones adjuntadas también se agregaron las figuras usadas respectivamente en estas. ■ Código generado de la clase AlertaCrecimiento. ■ Código generado de la clase Empleado. ■ Update readme. Se actualiza el archivo modelo/perfiles/readme indicando que no se utilizaron perfiles. ■ Agregar video demostrativo. Se añade el archivo evidencias/video-demo.mp4. ■ Update readme. ■ Update README.md. ■ Update readme no se usaron plantillas. Descripción del uso de plantillas.	■ 4004abd ■ d038efa ■ 5f2600c ■ 6cd895b ■ 714de4a ■ 1c8bac3 ■ f52e823 ■ dbbd3ab

10 Conclusiones

10.1 Valor del ciclo MDD para el PFC AquaGest

La aplicación del ciclo MDD con Modelio sobre el subsistema de Control de Estanques, Siembras y Seguimiento permite verificar, de forma concreta, que el modelo de dominio (clases **Estanque**, **Siembra**, **Empleado**, **SobrecargaSiembra**, **Grameo** y **AlertaCrecimiento**, con base en los RF-01 a RF-08 y RF-17 ya formalizados y validados con el stakeholder de la Compañía Biofina) es suficientemente preciso como para derivar de él una base de código estructural, reduciendo el riesgo de que la implementación posterior del PFC se desvíe de esos requisitos. La verificación directa del código generado (Sección 7.2) confirmó que Modelio Java Designer respeta fielmente la navegabilidad y la multiplicidad definidas en el modelo, lo que refuerza la confiabilidad del ciclo MDD como punto de partida para el desarrollo posterior del PFC, sin eximir de completar manualmente la lógica de negocio (Sección 7.3).

10.2 Aprendizajes obtenidos

La ejecución de la transformación modelo a código permitió comprobar que Modelio Java Designer genera automáticamente la estructura básica de las clases Java a partir del modelo UML, incluyendo atributos, identificador autogenerado, colecciones para las asociaciones y firmas de operación, respetando la navegabilidad y la multiplicidad definidas en cada relación del diagrama de clases (por ejemplo, la colección `List<Empleado>` generada en `Siembra.java` es consistente con la multiplicidad 1 a 0..* modelada para esa asociación). La ejecución del *roundtrip* controlado (Sección 8) permitió además comprobar en la práctica que Modelio Java Designer, con la configuración usada, no preserva código manual entre regeneraciones – un hallazgo directamente relevante para decidir cómo organizar el código del PFC en fases posteriores.

10.3 Limitaciones

El código generado por Modelio Java Designer corresponde a un esqueleto de implementación: los cuerpos de los métodos se generan vacíos y la lógica de negocio debe desarrollarse manualmente. La generación automática depende además de que la navegabilidad y la multiplicidad de cada asociación estén correctamente definidas en el modelo, ya que se trasladan de forma directa al código generado (por ejemplo, a colecciones `List<...>` en el extremo “muchos” de cada relación). Finalmente, esta demostración cubrió la generación *forward* y un *roundtrip* de un único ciclo (un cambio, una regeneración); no se evaluó el comportamiento del generador ante cambios más complejos o simultáneos en varias clases a la vez.

Referencias

- [1] D. C. Schmidt, “Guest editor’s introduction: Model-driven engineering,” *Computer*, vol. 39, no. 2, pp. 25–31, 2006.
- [2] A. Bucchiarone, J. Cabot, R. F. Paige, and A. Pierantonio, “Grand challenges in model-driven engineering: an analysis of the state of the research,” *Software and Systems Modeling*, vol. 19, no. 1, pp. 5–13, 2020.
- [3] Object Management Group, “Model driven architecture (mda) guide, revision 2.0,” Tech. Rep. ormsc/14-06-01, Object Management Group, 2014.
- [4] B. Selic, “The pragmatics of model-driven development,” *IEEE Software*, vol. 20, no. 5, pp. 19–25, 2003.
- [5] M. Brambilla, J. Cabot, and M. Wimmer, *Model-Driven Software Engineering in Practice*. Synthesis Lectures on Software Engineering, Morgan & Claypool Publishers, 2nd ed., 2017.
- [6] Object Management Group, “Meta object facility (MOF) 2.0 query/view/transformation specification, version 1.3,” Tech. Rep. formal/16-06-03, Object Management Group, 2016.
- [7] Object Management Group, “MOF model to text transformation language (MOFM2T), version 1.0,” Tech. Rep. formal/2008-01-16, Object Management Group, 2008.
- [8] Object Management Group, “OMG unified modeling language (OMG UML), version 2.5.1,” Tech. Rep. formal/17-12-05, Object Management Group, 2017.
- [9] N. Kahani, M. Bagherzadeh, J. R. Cordy, J. Dingel, and D. Varró, “Survey and classification of model transformation tools,” *Software and Systems Modeling*, vol. 18, no. 4, pp. 2361–2397, 2019.
- [10] J. Whittle, J. Hutchinson, and M. Rouncefield, “The state of practice in model-driven engineering,” *IEEE Software*, vol. 31, no. 3, pp. 79–85, 2014.
- [11] Castro Bajiña, Ariel Omar and Crespo Espinoza, Kleber Obed and Gamarra Araujo, Edhu Xavier and Pérez Ruiz, Carlos Andrés and Quintero Gende, Erick Jahir, “Aquagest – sistema para la gestión inteligente de una camaronera. especificación de requisitos de software (entrega 2, 1b).” https://github.com/EdhuXav/Proyecto_IR_Camaronera_CCGaPQ, 2026.
- [12] Wikipedia contributors, “Modelio.” <https://en.wikipedia.org/wiki/Modelio>, 2026. Consultado en julio de 2026. Licencia del núcleo: GPLv3; módulos: Apache License 2.0. Última versión estable referenciada: 5.4.01 (diciembre de 2023).
- [13] Modeliosoft, “Modelio open source – UML and BPMN free modeling tool.” <https://www.modelio.org/>, 2026. Consultado en julio de 2026.
- [14] Modeliosoft, “Modelio SD java – code generation and reverse engineering with UML.” <https://www.modeliosoft.com/en/products/modelio-sd-java.html>, 2026. Consultado en julio de 2026.
- [15] Modeliosoft, “Model-driven code generation and reverse engineering features.” <https://www.modeliosoft.com/en/features/model-driven-code-generation.html>, 2026. Consultado en julio de 2026.